

SESIÓN DE CLASE — Django REST Framework + API (FastAPI) + Proyecto Integrador

Sesión N° 1 — 2 Octubre de 2026 (Viernes) — Bloque 1: Django REST Framework

Instructor	César Augusto Moreno Mena	Centro	CESGE
Curso	Django REST Framework + API (FastAPI) + Proyecto Integrador	Año	2026

N°	Fecha	Día	Semana	Bloque	Tema de la sesión	Teoría	Ejemplo	Práctica
1	2 Octubre	Viernes	S1	Bloque 1: Django REST Framework	Introducción a Django	Arquitectura MTV (Model-Template-View), instalación, estructura de un proyecto y app en Django.	Creación de un proyecto Django y una app inicial, ejecución del servidor de desarrollo.	Los aprendices crean su primer proyecto Django con una app funcional.

1. Objetivo de la sesión

Al finalizar esta sesión, el aprendiz será capaz de **instalar Django**, explicar con sus propias palabras la **arquitectura MTV** (Model – Template – View), y **crear su primer proyecto Django con una aplicación (app) funcional** que responda en el navegador.

2. ¿Qué es Django y para qué sirve?

Django es un **framework** (marco de trabajo) para construir aplicaciones web usando Python. Un framework es como una casa prefabricada: ya trae las paredes, la instalación eléctrica y las tuberías resueltas, y tú solo te encargas de decorar y conectar las habitaciones según tu proyecto. Django ya trae resuelto: la conexión a bases de datos, el sistema de seguridad, el panel de administración, el manejo de formularios, entre otros. Por eso se dice que Django viene con "baterías incluidas".

□ Idea clave

No vas a programar un servidor web desde cero: Django ya te da todas las piezas. Tu trabajo es aprender *dónde* va cada pieza y *cómo* se conectan entre sí.

3. La arquitectura MTV (Model – Template – View)

Django organiza el código en tres partes, cada una con una responsabilidad clara. Esto se conoce como el patrón **MTV**:

M	Model (Modelo): representa los datos de tu aplicación. Es la parte que habla con la base de datos (ej. un modelo "Producto" que guarda nombre, precio y stock).
T	Template (Plantilla): representa lo que ve el usuario en el navegador (HTML). Es la parte visual.
V	View (Vista): es la lógica que conecta el Modelo con el Template: recibe la petición del usuario, busca o procesa los datos (Model) y decide qué respuesta enviar.

Una forma sencilla de recordarlo: el usuario pide algo (por ejemplo, "ver todos los productos") → la **View** recibe la petición → la View le pregunta al **Model** los datos → la View entrega esos datos a un **Template** (o, como veremos más adelante con Django REST Framework, los entrega directamente como JSON) → el usuario recibe la respuesta.

4. Instalación paso a paso

Sigue estos pasos exactamente en el orden indicado. Ejecuta cada comando en tu terminal (CMD, PowerShell, Terminal de Mac/Linux o la terminal integrada de VS Code).

Paso 1: Verificar que Python está instalado

```
python --version
```

Debes ver algo como `Python 3.11.x` o similar. Si el comando no funciona, prueba con `python3 --version`.

Paso 2: Crear un entorno virtual

Un entorno virtual es una "caja aislada" donde instalamos las librerías de este proyecto sin que se mezclen con las de otros proyectos.

```
python -m venv venv
```

Paso 3: Activar el entorno virtual

```
# En Windows (PowerShell o CMD):
venv\Scripts\activate

# En Mac / Linux:
source venv/bin/activate
```

Sabrás que está activado porque verás `(venv)` al inicio de la línea de tu terminal.

Paso 4: Instalar Django

```
pip install django
```

Paso 5: Verificar la instalación

```
django-admin --version
```

Si ves un número de versión (ej. `5.0.6`), ¡Django quedó instalado correctamente!

⚠ Error común

Si olvidas activar el entorno virtual (`(venv)` no aparece en la terminal) y luego instalas Django, este puede quedar instalado en el lugar equivocado del computador. Verifica siempre que `(venv)` esté visible antes de instalar cualquier librería.

5. Ejemplo guiado: creación del primer proyecto y app

Paso 1: Crear el proyecto

```
django-admin startproject config .
```

El punto final (`.`) le indica a Django que cree los archivos en la carpeta actual, en lugar de crear una subcarpeta adicional. Esto nos deja una estructura más limpia.

Paso 2: Explorar la estructura generada

```
proyecto/
├── manage.py
└── config/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

1	<code>manage.py</code> : el "control remoto" del proyecto. Con él ejecutamos el servidor, creamos apps, hacemos migraciones, etc.
2	<code>settings.py</code> : la configuración general del proyecto (apps instaladas, base de datos, idioma, zona horaria, etc.).
3	<code>urls.py</code> : el mapa de rutas principal del proyecto (qué URL activa qué funcionalidad).
4	<code>wsgi.py</code> / <code>asgi.py</code> : archivos técnicos usados para desplegar el proyecto en un servidor real (no los modificaremos por ahora).

Paso 3: Crear la primera app

En Django, un **proyecto** puede contener varias **apps**. Una app es un módulo que resuelve una funcionalidad específica (ej. "inventario", "usuarios", "ventas").

```
python manage.py startapp inventario
```

Esto crea una carpeta `inventario/` con archivos como `models.py`, `views.py`, `admin.py` y `migrations/`. Los usaremos en las próximas sesiones.

Paso 4: Registrar la app en el proyecto

Abre `config/settings.py` y busca la lista `INSTALLED_APPS`. Agrega el nombre de tu app al final:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'inventario', # <-- nuestra app  
]
```

⚠ Error común

Si olvidas este paso, Django **no reconocerá** los modelos ni las funcionalidades de tu app, aunque la carpeta exista. Siempre que crees una app nueva, este es el segundo paso obligatorio (después de `startapp`).

Paso 5: Crear una vista simple

Abre `inventario/views.py` y escribe:

```
from django.http import HttpResponse  
  
def inicio(request):  
    return HttpResponse("Hola, soy la app de inventario")
```

Paso 6: Conectar la vista a una URL

Crea el archivo `inventario/urls.py` (no existe por defecto, lo creas tú):

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.inicio, name='inicio'),  
]
```

Ahora abre `config/urls.py` (el del proyecto) e incluye las rutas de la app:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('inventario/', include('inventario.urls')),
]
```

Paso 7: Ejecutar el servidor de desarrollo

```
python manage.py runserver
```

Abre tu navegador en `http://127.0.0.1:8000/inventario/` y deberías ver el texto "Hola, soy la app de inventario".

6. Ejercicio práctico (para hacer en clase)

Ahora te toca a ti. Sigue estos pasos sin copiar directamente del ejemplo anterior, escribiendo el código con tus propias manos (esto ayuda a que el aprendizaje se fije mejor):

1. Crea un nuevo proyecto llamado `mi_primer_proyecto`.
2. Crea una app llamada `saludos`.
3. Regístrala en `INSTALLED_APPS`.
4. Crea una vista que devuelva el texto: `"Bienvenido al curso de Django, [tu nombre]"` (reemplaza `[tu nombre]` por tu nombre real).
5. Conecta esa vista a la ruta `/saludos/`.
6. Levanta el servidor y verifica en el navegador que el mensaje aparezca correctamente.
7. **Reto adicional (opcional):** agrega una segunda vista en la ruta `/saludos/adios/` que devuelva un mensaje de despedida.

7. Errores comunes y cómo evitarlos

x	" ModuleNotFoundError: No module named 'django' ": significa que el entorno virtual no está activado o Django no se instaló dentro de él. Activa <code>venv</code> y vuelve a instalar.
x	" Página no encontrada (404) ": revisa que la ruta en <code>urls.py</code> de la app esté incluida en <code>urls.py</code> del proyecto con <code>include()</code> .
x	" Puerto en uso (Error: That port is already in use) ": otro proceso está usando el puerto 8000. Ejecuta <code>python manage.py runserver 8001</code> para usar otro puerto.
x	La app no aparece / da error de configuración : revisa que el nombre en <code>INSTALLED_APPS</code> coincida exactamente con el nombre de la carpeta de la app.

8. Resumen de la sesión (checklist de aprendizaje)

☐	Sé instalar Django dentro de un entorno virtual.
☐	Puedo explicar qué es Model, Template y View con mis propias palabras.
☐	Sé crear un proyecto Django con <code>startproject</code> .
☐	Sé crear una app con <code>startapp</code> y registrarla.
☐	Sé crear una vista simple y conectarla a una URL.
☐	Puedo ejecutar el servidor de desarrollo y ver el resultado en el navegador.

Próxima sesión (5 de octubre): aprenderemos a crear Modelos y a usar el ORM de Django para guardar información en una base de datos real.